BACKTRACKING

FAANG Master Guide — Java Edition

💡 WHY BACKTRACKING?

Backtracking is a controlled DFS search over a **State Space Tree**. It explores candidate solutions and **backtracks (undoes choice)** as soon as a path violates constraints!

💡 THE 3-STEP TRINITY

1. **CHOOSE:** Add candidate to path (`list.add(val)`).
2. **EXPLORE:** Recurse to next level (`dfs(...)`).
3. **UNDO:** Remove candidate (`list.remove(list.size() - 1)`).

1 3 CORE CATEGORIES

- **Subsets (2^N):** Pick or skip each element (`start` pointer increments).
- **Combinations ($N! / K!$):** Choose K elements out of N (`start` pointer).
- **Permutations ($N!$):** Order matters (`visited[]` array or element swapping).

2 VISUALIZING STATE SPACE TREE

Subsets of [1, 2]:

```
[]
├── [1]
│   └── [1, 2]
└── [2]
Total Subsets = 2^N = 4 ({}, {1}, {1,2}, {2})
```

📚 PREREQUISITES & COMPLEXITY REASONING

Category	Time Complexity	Auxiliary Space
Subsets (2^N)	**O(N · 2^N)**	**O(N) recursion stack**
Combinations (nCr)	**O(K · nCr)**	**O(K) stack**
Permutations ($N!$)	**O(N · N!)**	**O(N) stack**
N-Queens / Sudoku	**O(N!)**	**O(N) board state**

⚡ CLUES THAT SCREAM BACKTRACKING

1. *"Return all possible combinations / subsets / permutations"*
2. *"Duplicates allowed in input, output must contain UNIQUE results"* → Sort + Skip duplicate adjacent elements
3. *"Place N queens on board / solve Sudoku"* → Constraint Validation Grid DFS

PATTERN 1 SUBSETS I (Power Set Generation) e.g. Subsets (LC 78)

WHEN TO USE

Generate all 2^N subsets. Every node in the state space tree is a valid subset snapshot!

TEMPLATE — LC 78

```
public List<List<Integer>> subsets(int[] nums) {
  List<List<Integer>> res = new ArrayList<>();
  dfs(res, new ArrayList<>(), nums, 0);
  return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trac
k, int[] nums, int start) {
  res.add(new ArrayList<>(track));
  for (int i = start; i < nums.length; i++) {
    track.add(nums[i]);
    dfs(res, track, nums, i + 1);
    track.remove(track.size() - 1);
  }
}
```

PATTERN 2 SUBSETS II (Duplicates in Input) e.g. Subsets II (LC 90)

WHEN TO USE

Input array contains duplicates. Must sort and skip duplicate branch at same level (`i > start`).

TEMPLATE — LC 90

```
public List<List<Integer>> subsetsWithDup(int[] nums) {
  Arrays.sort(nums);
  List<List<Integer>> res = new ArrayList<>();
  dfs(res, new ArrayList<>(), nums, 0);
  return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trac
k, int[] nums, int start) {
  res.add(new ArrayList<>(track));
  for (int i = start; i < nums.length; i++) {
    if (i > start && nums[i] == nums[i - 1]) continue;
    track.add(nums[i]);
    dfs(res, track, nums, i + 1);
    track.remove(track.size() - 1);
  }
}
```

PATTERN 5 PERMUTATIONS I (Order Matters, Distinct Elements) e.g. Permutations (LC 46)

VISITED TRACKING

Order matters, so loop from `0` to `N-1` at every level. Track used elements via `boolean[] visited` or `track.contains()`.

TEMPLATE — LC 46

```
public List<List<Integer>> permute(int[] nums) {
  List<List<Integer>> res = new ArrayList<>();
  dfs(res, new ArrayList<>(), nums, new boolean[nums.lengt
h]);
  return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trac
k, int[] nums, boolean[] vis) {
  if (track.size() == nums.length) { res.add(new ArrayList<>
(track)); return; }
  for (int i = 0; i < nums.length; i++) {
    if (vis[i]) continue;
    vis[i] = true; track.add(nums[i]);
    dfs(res, track, nums, vis);
    track.remove(track.size() - 1); vis[i] = false;
  }
}
```

PATTERN 6 PERMUTATIONS II (Order Matters + Input Duplicates) e.g. Permutations II (LC 47)

VISITED PRUNING CONDITION

Sort array first. Skip duplicate if `nums[i] == nums[i-1]` AND `!visited[i-1]`.

TEMPLATE — LC 47

```
public List<List<Integer>> permuteUnique(int[] nums) {
  Arrays.sort(nums);
  List<List<Integer>> res = new ArrayList<>();
  dfs(res, new ArrayList<>(), nums, new boolean[nums.lengt
h]);
  return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trac
k, int[] nums, boolean[] vis) {
  if (track.size() == nums.length) { res.add(new ArrayList<>
(track)); return; }
  for (int i = 0; i < nums.length; i++) {
    if (vis[i] || (i > 0 && nums[i] == nums[i - 1] && !vis[i
- 1])) continue;
    vis[i] = true; track.add(nums[i]);
    dfs(res, track, nums, vis);
    track.remove(track.size() - 1); vis[i] = false;
  }
}
```

📊 COMPLETE BACKTRACKING PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
78	Subsets	Subsets I	O(N · 2^N)	O(N)	M
90	Subsets II	Subsets II Dup Pruning	O(N · 2^N)	O(N)	M
39	Combination Sum	Unlimited Reuse	O(2^T)	O(T)	M
40	Combination Sum II	Single Use + Dup Pruning	O(2^N)	O(N)	M
46	Permutations	Visited Permutations	O(N · N!)	O(N)	M
47	Permutations II	Permutations Dup Pruning	O(N · N!)	O(N)	M
79	Word Search	Grid DFS Backtracking	O(N · 4^L)	O(L)	M
131	Palindrome Partitioning	Substring Partitioning	O(N · 2^N)	O(N)	M
51	N-Queens	Constraint Grid DFS	O(N!)	$O(N^2)$	H

LC 78 · Subsets

MEDIUM

Pattern: Subsets I
Key Insight: Snapshot copy at every node.

```
public List<List<Integer>> subsets(int[] nums) {
  List<List<Integer>> res = new ArrayList<>();
  dfs(res, new ArrayList<>(), nums, 0); return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trk, int[] n, int s) {
  res.add(new ArrayList<>(trk));
  for (int i = s; i < n.length; i++) {
    trk.add(n[i]); dfs(res, trk, n, i + 1); trk.remove(trk.size() - 1);
  }
}
```

LC 90 · Subsets II

MEDIUM

Pattern: Subsets II Dup Pruning
Key Insight: Sort + `i > start && nums[i] == nums[i-1]`.

```
public List<List<Integer>> subsetsWithDup(int[] nums) {
  Arrays.sort(nums); List<List<Integer>> res = new ArrayList<>();
  dfs(res, new ArrayList<>(), nums, 0); return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trk, int[] n, int s) {
  res.add(new ArrayList<>(trk));
  for (int i = s; i < n.length; i++) {
    if (i > s && n[i] == n[i - 1]) continue;
    trk.add(n[i]); dfs(res, trk, n, i + 1); trk.remove(trk.size() - 1);
  }
}
```

LC 39 · Combination Sum

MEDIUM

Pattern: Unlimited Element Reuse
Key Insight: Pass `i` to next recursive call.

```
public List<List<Integer>> combinationSum(int[] cand, int target) {
  List<List<Integer>> res = new ArrayList<>();
  dfs(res, new ArrayList<>(), cand, target, 0); return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trk, int[] c, int rem, int s) {
  if (rem < 0) return;
  if (rem == 0) { res.add(new ArrayList<>(trk)); return; }
  for (int i = s; i < c.length; i++) {
    trk.add(c[i]); dfs(res, trk, c, rem - c[i], i); trk.remove(trk.size() - 1);
  }
}
```

LC 40 · Combination Sum II — MEDIUM

Pattern: Single Use + Dup Pruning
Key Insight: Sort + `i + 1` + skip duplicate.

```
public List<List<Integer>> combinationSum2(int[] cand, int target) {
  Arrays.sort(cand); List<List<Integer>> res = new ArrayList<>();
  dfs(res, new ArrayList<>(), cand, target, 0); return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trk, int[] c, int rem, int s) {
  if (rem < 0) return;
  if (rem == 0) { res.add(new ArrayList<>(trk)); return; }
  for (int i = s; i < c.length; i++) {
    if (i > s && c[i] == c[i - 1]) continue;
    trk.add(c[i]); dfs(res, trk, c, rem - c[i], i + 1); trk.remove(trk.size() - 1);
  }
}
```

LC 46 · Permutations — MEDIUM

Pattern: Visited Boolean Permutations
Key Insight: Loop $0..N-1$, skip if `visited[i]`.

```
public List<List<Integer>> permute(int[] nums) {
  List<List<Integer>> res = new ArrayList<>();
  dfs(res, new ArrayList<>(), nums, new boolean[nums.length]); return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trk, int[] n, boolean[] vis) {
  if (trk.size() == n.length) { res.add(new ArrayList<>(trk)); return; }
  for (int i = 0; i < n.length; i++) {
    if (vis[i]) continue;
    vis[i] = true; trk.add(n[i]); dfs(res, trk, n, vis);
    trk.remove(trk.size() - 1); vis[i] = false;
  }
}
```

LC 79 · Word Search — MEDIUM

Pattern: Grid DFS Backtracking
Key Insight: Temp replace `board[r][c] = '#'`.

```
private boolean dfs(char[][] b, String w, int r, int c, int idx) {
  if (idx == w.length()) return true;
  if (r<0||r≥b.length||c<0||c≥b[0].length||b[r][c] ≠ w.charAt(idx)) return false;
  char tmp = b[r][c]; b[r][c] = '#';
  for (int[] d : DIRS) if (dfs(b, w, r + d[0], c + d[1], idx + 1)) return true;
  b[r][c] = tmp; return false;
}
```

LC 131 · Palindrome Partitioning — MEDIUM

Pattern: Substring Partitioning
Key Insight: Recurse if `isPalindrome(s, start, i)`.

```
private void dfs(List<List<String>> res, List<String> trk, String s, int start) {
  if (start == s.length()) { res.add(new ArrayList<>(trk)); return; }
  for (int i = start; i < s.length(); i++) {
    if (isPal(s, start, i)) {
      trk.add(s.substring(start, i + 1));
      dfs(res, trk, s, i + 1);
      trk.remove(trk.size() - 1);
    }
  }
}
```

LC 51 · N-Queens — HARD

Pattern: Constraint Grid DFS
Key Insight: Validate row, column, and diagonals.

```
private void dfs(List<List<String>> res, char[][] b, int row, int n) {
  if (row == n) { res.add(construct(b)); return; }
  for (int col = 0; col < n; col++) {
    if (isValid(b, row, col, n)) {
      b[row][col] = 'Q'; dfs(res, b, row + 1, n); b[row][col] = '.';
    }
  }
}
```

LC 17 · Phone Number Combinations — MEDIUM

Pattern: Digit Keypad Mapping
Key Insight: Map digits to letters string array.

```
private void dfs(List<String> res, StringBuilder sb, String digits, int idx, String[] map) {
  if (idx == digits.length()) { res.add(sb.toString()); return; }
  String letters = map[digits.charAt(idx) - '0'];
  for (char c : letters.toCharArray()) {
    sb.append(c); dfs(res, sb, digits, idx + 1, map); sb.deleteCharAt(sb.length() - 1);
  }
}
```

🔍 DRY RUN — Subsets ([1, 2])

Step	Track State	Action	Snapshot Added to Result
1	[]	`res.add([])`	`[]`
2	[1]	Add 1, recurse `start=1` -> `res.add([1])`	`[1]`
3	[1, 2]	Add 2, recurse `start=2` → `res.add([1, 2])`	`[1, 2]`
4	[1]	Pop 2 → Return to level 1	-
5	[]	Pop 1 → Loop `i=1` (add 2) → `res.add([2])`	`[2]` ✅

📐 MATHEMATICAL PROOF — 2^N SUBSETS DERIVATION

Claim: An array of N distinct elements produces exactly 2^N subsets.

Proof: For each element $x_i \in [1..N]$, we have binary choices: INCLUDE x_i or EXCLUDE x_i. Total subsets $= 2 \times 2 \times \dots \times 2 = 2^N$. Snapshot copy takes O(N) time → **O(N · 2^N) Time!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Write Subsets I in 3m
☐ Code Subsets II duplicate pruning
☐ Write Combination Sum (element reuse `i`)
☐ Code Permutations with `boolean[] visited`
☐ Write Permutations II duplicate check
☐ Code Word Search 2D Grid DFS
☐ Write N-Queens board placement DFS
☐ Prove why Subsets is O(N · 2^N)

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Always Make a Copy for Result
Always add a NEW copy to result list (`res.add(new ArrayList<>(track))`), NOT the original reference `res.add(track)`!

Rule 2 — Always Undo the Last Element
After the recursive call returns, immediately undo the choice: `track.remove(track.size() - 1)`!